

1 LED マトリクスでの BPM 表示（変更後）

1.0.1 ファイル構成

表 1 BPM 表示機能のファイル構成

ファイル名	概要
Display.ino	WiFi 接続・UDP 受信開始の管理
Display.cpp	表示文字の独自フォント定義, LED マトリクスへの描画処理, UDP 受信処理
Display.h	定数定義, 関数宣言, 外部参照

Display.ino は WiFi 接続と UDP 受信の開始を管理する。描画処理および通信処理はすべて Display.cpp に集約する。これは Arduino_LED_Matrix の表示バッファがヘッダ内で static 定義されており、include する翻訳単位ごとに別実体となるため、begin() と loadFrame() を同一ファイルから呼び出す必要があることによる。

ファイル構成について説明する。また、表 1 に BPM 表示機能のファイル構成を示している。はじめに、Display.ino ファイルでは、表 1 に示すように WiFi 接続と UDP 接続を行う。今回、指揮機と中継機間の通信形式としては、高速通信を目的とし、UDP 形式で情報を受け取る。

次に、Display.cpp および.h ファイルでは、表示文字の独自フォント定義、マトリクスへの描画処理を行う。ここで、今回利用する Arduino のマトリクスは縦 12 横 8 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 を以下に示す。

```
{1,1,1,  
1,0,1,  
1,0,1,  
1,0,1,  
1,1,1}
```

また、LED マトリクスの描画処理は、受信したデータを文字列から整数に変換し、整数を各桁で分割、それぞれを定義した配列に当てはめることで行う。描画自体はマト

リクスパッケージの機能を用いて、配列を表現する.

1.1 クラス図

変更後のモジュール構成を以下に示す.

Display.ino

- ・ `setup()`: `displaySetup()` の呼び出し, WiFi 接続, UDP 受信開始
- ・ `loop()`: `checkUDP()` の呼び出し

Display.cpp

- ・ `displaySetup()`: `matrix.begin()` およびウォームアップ描画
- ・ `checkUDP()`: UDP パケットの確認, ヘッダ判定, BPM 更新・消灯処理
- ・ `displayBPM()`: BPM 値を桁分解し, 中央寄せでフレームバッファに描画
- ・ `drawDigit()`: 1 桁の数字を `font3x5[]` からフレームバッファに描画
- ・ `clearDisplay()`: LED マトリクスを全消灯し `currentBPM` をリセット
- ・ `font3x5[10][15]`: 独自 3×5 数字フォント (0~9)
- ・ `frameBuffer[8][12]`: 描画用共有フレームバッファ
- ・ `currentBPM`: 直近に表示中の BPM 値 (更新検知用)

Display.h

- ・ 定 数 定 義: `BPM_STEP_1~ 5`, `LOCAL_PORT`, `HEADER_BPM`, `HEADER_END`,
`FONT_WIDTH/HEIGHT`, `MATRIX_ROWS/COLS`
- ・ 関数宣言: 上記 5 関数
- ・ 外部参照: `font3x5`, `matrix`, `Udp`

1.2 関数設計

表 2 関数一覧

関数名	所属	概要
displaySetup	Display.cpp	matrix.begin() および初回描画のウォームアップ
checkUDP	Display.cpp	UDP パケットの確認, ヘッダ判定, BPM 更新・消灯処理
displayBPM	Display.cpp	BPM 値を桁分解し, 中央寄せでフレームバッファに描画
drawDigit	Display.cpp	1 桁の数字を font3x5[] からフレームバッファに描画
clearDisplay	Display.cpp	LED マトリクスを全消灯し currentBPM をリセット

checkUDP 関数は, `Udp.parsePacket()` でパケットの到着を監視する. パケットが届いている場合, `uint8_t buffer[2]` としてバイナリ形式で読み取り, `buffer[0]` をヘッダ文字, `buffer[1]` を段階番号として処理する. ヘッダが'E' の場合は `clearDisplay()` を呼び出す. ヘッダが'B' の場合は `bpmTable[]` を参照して BPM 値に変換し, 値が変化した場合のみ `displayBPM()` を呼び出す.

displayBPM 関数は, BPM 数値を桁ごとに分割し, `drawDigit()` を用いてフレームバッファに描画した後, `uint32_t[3]` 形式にビットパックして `matrix.loadFrame()` で LED に反映する.

clearDisplay 関数は, `currentBPM` を初期値 (-1) にリセットし, 全消灯フレーム (`uint32_t blank[3] = {0, 0, 0}`) を `matrix.loadFrame()` で描画する.

displaySetup 関数は, `matrix.begin()` を呼び出した後, 消灯フレームを一度描画 (ウォームアップ) してから 200ms 待機する. `begin()` 直後の最初の `loadFrame()` は反映されない場合があるための対策である.